

NERV

A hardware neural network implementation of a health state detector

ELC2403: Electronic Circuits

Islam Tarek Moawad



NERV

A hardware neural network implementation of
a health state detector

by

Islam Tarek Moawad

Instructors: Prof. Roaa Mobarak
Prof. Fathi Zaki
Faculty: Faculty of Engineering Capital University,
Electronics and Communication Department.



Preface

I would like to mention the different nature of this project compared to others; it aims to prove a theory and to be a simplified model of a principle that can be followed to implement larger systems. and I would also like to say:

In our current era, under the capitalist system that dominates all aspects of life, and due to the rapid changes in the world around us resulting from the information technology revolution, we are in a race with no finish line to keep pace with developments in all technological sectors. And one development that cannot be ignored is the sudden advancement in artificial intelligence technologies, which emerged like a flood, engulfing all technological and non-technological sectors and bringing up a transformation that even the first industrial revolution did not achieve.

This project, and my choice to undertake it alone, is my way to express how much pressure our generation has to endure. Especially in engineering education, where the current assessment system forces students to memorize concepts rather than truly understand them or think creatively, and where an engineer's value is measured primarily by their ability to perform well on exams. As a result, learning becomes less about developing insight and innovation, and more about competing to accumulate the highest possible scores.

Summary

Neural networks are the backbone of artificial intelligence (AI), enabling it to make decisions. In most AI systems, this network is designed using software implementation. In this project, we will build a network that is equivalent to logical systems to check the stability or instability of five vital signs of a patient and determine their condition. The network will be built using analog hardware implementation by representing the neurons and other elements with operational amplifiers.

Before designing the network, we first need to design it at a low level on the computer. Therefore, I put the possibilities into a database and trained a simple model that DeepSeek  built in python. The model was then displayed as a visual neural network to determine the strengths of the connections between the neurons.

Following the low-level design comes the high-level design, in which the neurons and their connections are represented by an equivalent circuit consisting of two operational amplifiers and weights, in which resistors were used to represent the positive weights and Negative Impedance Converter circuits to represent the negative weights.

The connections between the neurons were designed as bus connections, and the system was powered by a power grid consisting of power lines supplied from three different voltage sources. The circuit was designed and simulated using Proteus and LTspice.

A very simple system was designed, taking into account physical, monetary, and spatial limitations, but fundamental principles and rules were established to determine how this system can be represented within the VLSI manufacturing using Cadence Virtuoso, to build it as an IC.

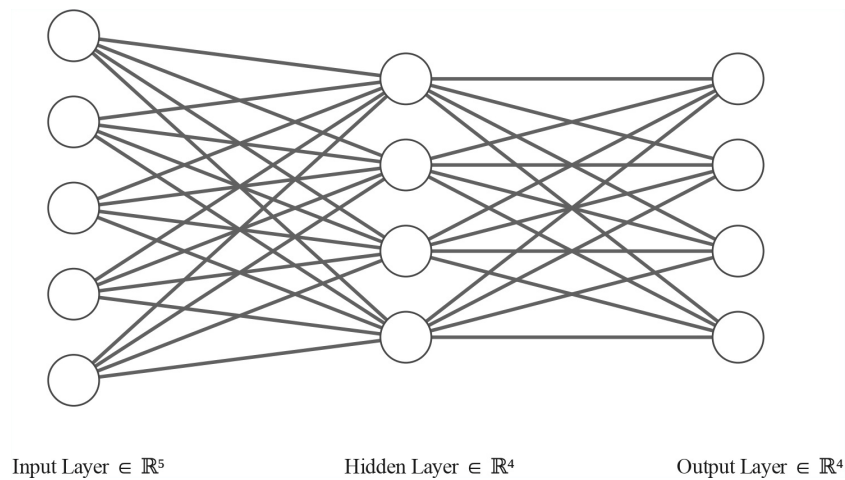


Figure 1: Neural network

Contents

Preface	i
Summary	ii
1 Introduction	1
1.1 Neural Network	1
1.2 Artificial Neurons	2
2 Software Implementation	3
2.1 Overview	3
2.2 Code and neural network plotting	4
2.2.1 model creating and training code	4
2.2.2 plotting the neural network	4
3 Hardware implementation	5
3.1 Neurons	5
3.2 wights	6
3.3 System	6
3.4 Schematic	7
4 Improvements	8
4.1 Neural network structure	8
4.2 Negative weights	8
4.3 Rebuilding the neuron	8
4.4 Input switches	8
5 Recalculating	9
6 Resistance Finder	10
6.1 Resistance Finder website	10
6.2 equivalent resistors connections	11
7 Hardware Re-implementation	12
7.1 schematic	12
7.2 layout	13
7.3 KiCAD interface	13
7.4 bottom layer	14
7.5 Isometric View	14
8 Gallery	15
9 Conclusion	16

Introduction

1.1. Neural Network

first of all we need to understand the concept of neural network.

A neural network is a system contains a several layers, each layer (except input layer) consists of units called artificial neurons which connected with the neurons in the two surrounding layer's.

this whole system takes inputs and produce outputs just like any other systems.

the thing that makes it special is it can learn how the output should be by practicing on similar data.

And this happens by an algorithm that calculate the wight of each connection based on the training data.

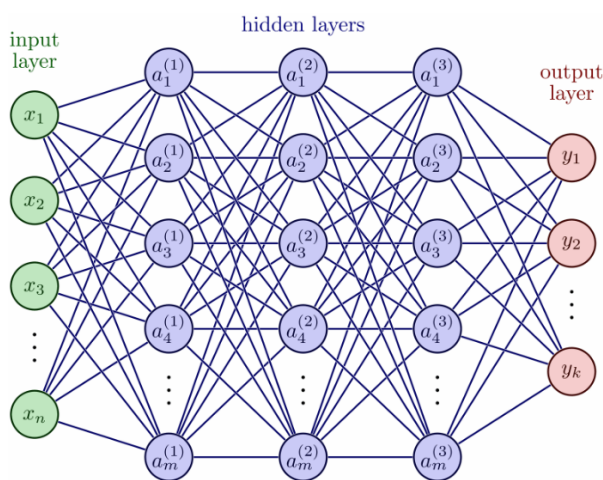


Figure 1.1: Neural network

We don't need to delve into neural networks because it's a broad topic and because this project doesn't require a deep understanding of them.

1.2. Artificial Neurons

Artificial Neurons as we mentioned before is the bricks that builds a neural network, but what it is?

Artificial neurons are an attempt to mimic natural neurons in the human brain.

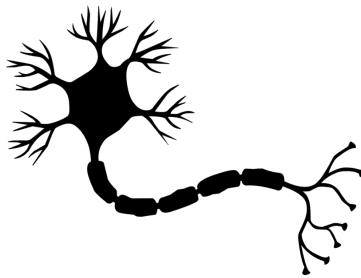


Figure 1.2: Natural neuron

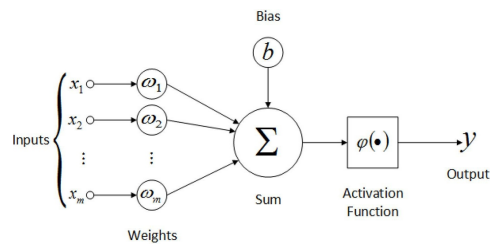


Figure 1.3: Artificial neuron

How it works?

both artificial and natural neurons have inputs and outputs terminals that connect them with other neurons, their inputs comes from the connection between the neuron itself and nodes in the layer before.

Each connection have it's wight, and it can be positive,negative or even zero (means no connection).

when a neuron or an input node are activated they send one in their output and each neuron in the following layer takes this One multiplied with the wight of the connection between them.

the neuron is activated when the sum of it's inputs are greater than a threshold value which will be Zero in our system.

2

Software Implementation

2.1. Overview

for our system, it should take a 5 inputs which represent the stability of a five vital graphs, if it's stable the sensor produces a 0 and if not it will be 1.

the output of the system will be 4 outputs each of them represents the state the patient is in in real-time. their will not be any case that two outputs are activated at the same time, so we have 4 probabilities of output, and 32 for inputs.

The model digit's meaning on the top par of Figure[2.1], is the input's and output's codes, and it's as shown in following table

code	Input (sensor)	Output (state)
1		stable
2		tired
3		unstable
4		emergency
5		

From the table we can see that the main function of this system to give a different output for each threshold value of summations. After writing all probabilities by hand on excel Figure[2.1], it's time to train a model. I asked DeepSeek to write a Deep-Learning model for me that consists of 5 inputs, 1 hidden layer with 4 neurons and 4 outputs.

It generated a code I wrote in Visual Studio, and used it to generate a .keras file of the model, which it's accuracy was 100% because it was a few data to train on.

I used the .keras file to get the wights of all connections between the neurons by a code DeepSeek wrote for me in python too.

n	1	2	3	4	5	1	2	3	4
1	0	0	0	0	0	1			
2	0	0	0	0	1	1			
3	0	0	0	1	0	1			
4	0	0	0	1	1		1		
5	0	0	1	0	0	1			
6	0	0	1	0	1		1		
7	0	0	1	1	0		1		
8	0	0	1	1	1			1	
9	0	1	0	0	0	1			
10	0	1	0	0	1		1		
11	0	1	0	1	0		1		
12	0	1	0	1	1			1	
13	0	1	1	0	0		1		
14	0	1	1	0	1			1	
15	0	1	1	1	0			1	
16	0	1	1	1	1				1
17	1	0	0	0	0	1			
18	1	0	0	0	1		1		
19	1	0	0	1	0		1		
20	1	0	0	1	1			1	
21	1	0	1	0	0		1		
22	1	0	1	0	1			1	
23	1	0	1	1	0			1	
24	1	0	1	1	1				1
25	1	1	0	0	0		1		
26	1	1	0	0	1			1	
27	1	1	0	1	0			1	
28	1	1	0	1	1				1
29	1	1	1	0	0		1		
30	1	1	1	0	1				1
31	1	1	1	1	0				1
32	1	1	1	1	1				1

Figure 2.1

3

Hardware implementation

3.1. Neurons

Before we start designing and implementing the system as an electronic circuit we need to know how to represent a single neuron using Operational Amplifier.

as we mentioned before we use the sum of the signals entering the neuron to activate it if it was higher than 0, so we need two op amps, a summing amplifier with a threshold amplifier after it.

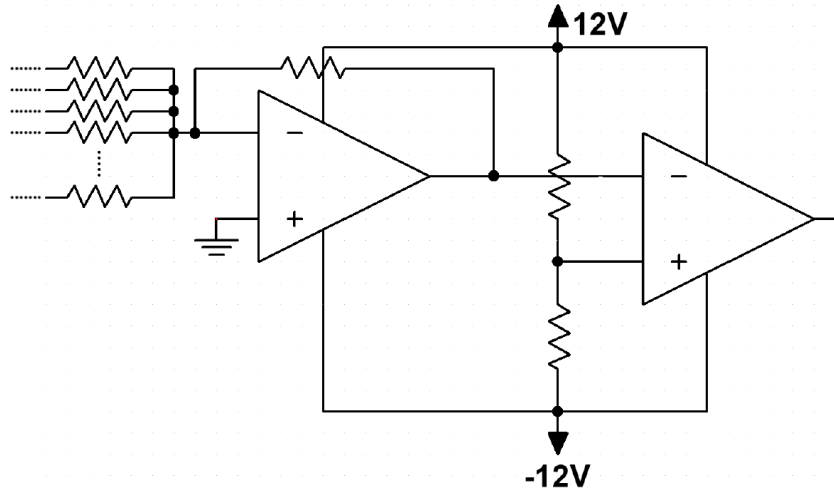


Figure 3.1

each connection weight is $\frac{R_{a|b.Ref}}{R_{(a-1|n) \rightarrow (a|b)}}$ and the equation for the output of each neuron is

$$s_{a|b} = 12 \times \varphi \left(\left(\sum_{n=0}^{N_{a-1}} \frac{R_{a|b.Ref}}{R_{(a-1|n) \rightarrow (a|b)}} \times s_{a-1|n} \right) + B_{a|b} \right)$$

I actually made this equation form Figure[1.3] & Figure[3.1]
there is no present of it in any reference I road.

let's explain it.

$s_{a|b}$ is the output signal of b-th neuron on the a-th layer. which is the $12 \times \varphi(.)$ of a function.

such that:

$$\varphi(x) = \begin{cases} 0, & x \leq 0 \\ 1, & x > 0 \end{cases} \quad (3.1)$$

the function inside the φ consists of two parts, the summator and the bias voltage that is the threshold voltage, in this system the threshold voltage is always zero so there is no need for the term $B_{a|b}$.

3.2. wights

for positive weights we will use a normal resistors, for negative weight we need an electronic circuit called inverse impedance converter Figure[3.2]. the impedance of the circuit in Figure[3.2] equals $-r$,

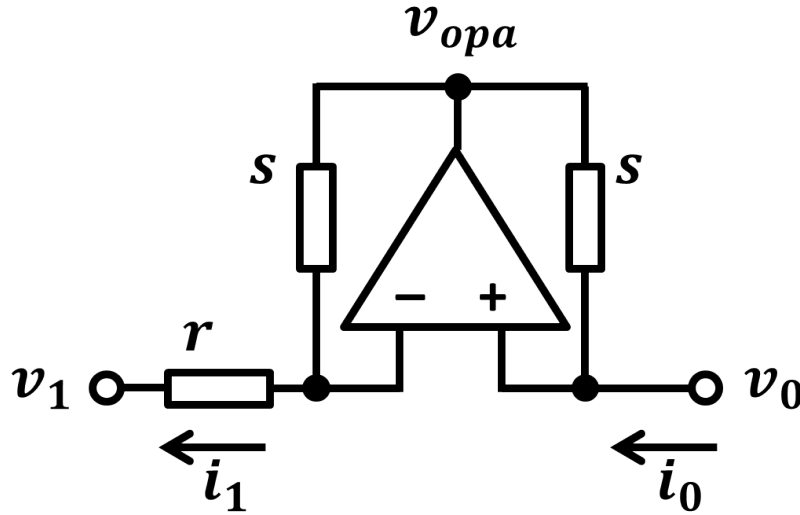


Figure 3.2: Negative impedance converter

and the optimum value of S will come out to us by doing a parametric sweep for a transient analyses on LTspice for the value s from 1 to $1k \Omega$. after doing the S -value sweep transient analyses it seems like the lower the value of S , the more liner the negative impedance is Figure[3.3]. so we will use the value of S to be as low as it can be.

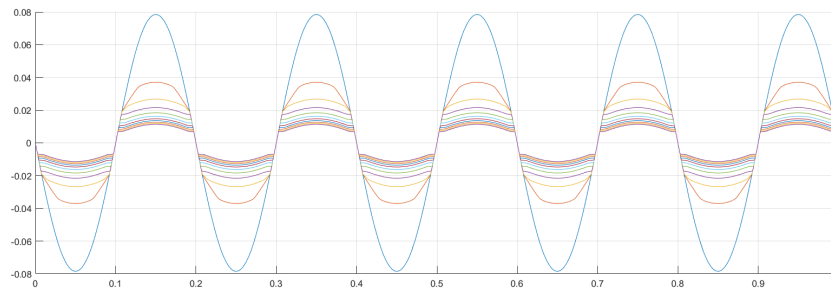
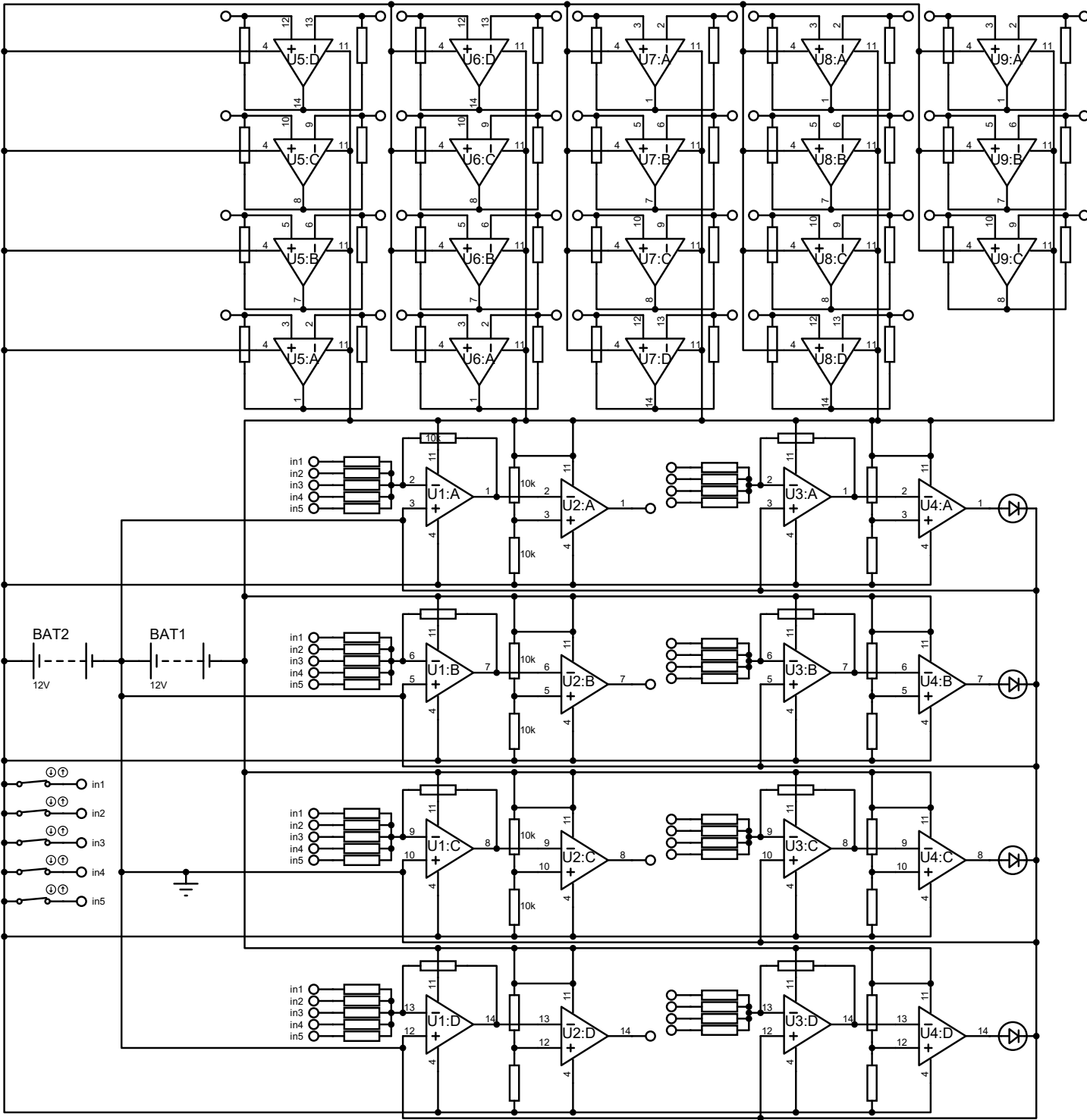


Figure 3.3: current in r for each value of S

3.3. System

we will use lm324 for all operational amplifiers in our system. and for design and simulation we will use Proteus 8 Professional and LTspice. we will implement this system on a test board and for power we will use 3 different voltages from the power supply, we will get our inputs from an esp32-s3, and for outputs we will use and LED to monitor them. the lower matrix is the Negative Impedance converters, the four switches is the five inputs, the four LEDs is the system's output, in the middle we can see the two layers. For powering we used a Three-voltage supply ($V+$, $V-$, GND)

3.4. Schematic



4

Improvements

4.1. Neural network structure

after several tries, I found that the minimum neurons that can give a reliable results is a 5 inputs, 2 neuron on the hidden layer, and 4 output neurons, that's a 6 neurons system and it's much much more reliable and easier for calculation ,simulation and implementation.

4.2. Negative weights

Instead of using a negative weights, we can just subtract the the weighs that assigned to be negative from the positive weights.

the gain is not a thing to worry about in this system, we must be much more considered about the ratios between the resistors to get the correct weights and biases values.

4.3. Rebuilding the neuron

each neuron has activation function, witch has been translated into the threshold amplifier, by understanding how really the threshold amplifier works, I found that the main purpose of the two resistors in it is to function as a voltage divider and give us the value between V_{++} and V_{--} , but we have a wire that has this value and it's fixed, so we can connect this wire and there is no need for these two resistors.

4.4. Input switches

the circuit above had a major problem, when the switches are open, the inputs become a flouting points. and according to the simulation on proteus, the voltage on them is nearly equal to 12, that's like they are still shorted.

so we need to find another switch that prevents the problem of floating points. and I've found it and it called SPDT Switch



Figure 4.1: SPDT Switch

5

Recalculating

after rebuilding the new structure of the system, I asked Grok to build a program that extracts the value of each connection weight and bias for each neuron, and this is the result shown in Figure[5.1].

```

2025-12-17 21:28:31.427812: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2025-12-17 21:28:43.861538: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2025-12-17 21:28:50.287983: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
Model Summary:
Model: "sequential"

```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 2)	12
dense_1 (Dense)	(None, 4)	12

```

Total params: 74 (300.00 B)
Trainable params: 24 (96.00 B)
Non-trainable params: 0 (0.00 B)
Optimizer params: 50 (204.00 B)

Hidden Layer Weights (from inputs to hidden neurons):
Hidden Neuron 1: [-1.827507 -1.8309762 -1.8314332 0.19155298 -1.828446 ]
Hidden Neuron 2: [2.2558564 2.2512004 2.2537787 2.2520714 2.254901 ]

Hidden Layer Biases:
[ 1.5911049 -3.9507325]

Output Layer Weights (from hidden to output neurons):
Output Neuron 1: [ 3.3496714 -8.121808 ]
Output Neuron 2: [-4.9300394 -1.3065317]
Output Neuron 3: [-3.5006778 0.2452718]
Output Neuron 4: [-2.5571008 1.4358329]

Output Layer Biases:
[ 1.699191 1.1651057 -1.299715 -5.7519016]
Press any key to continue . . .

```

Figure 5.1: wights and biases values

the values shown in the terminal is the weights and the biases of the trained .keras model file.

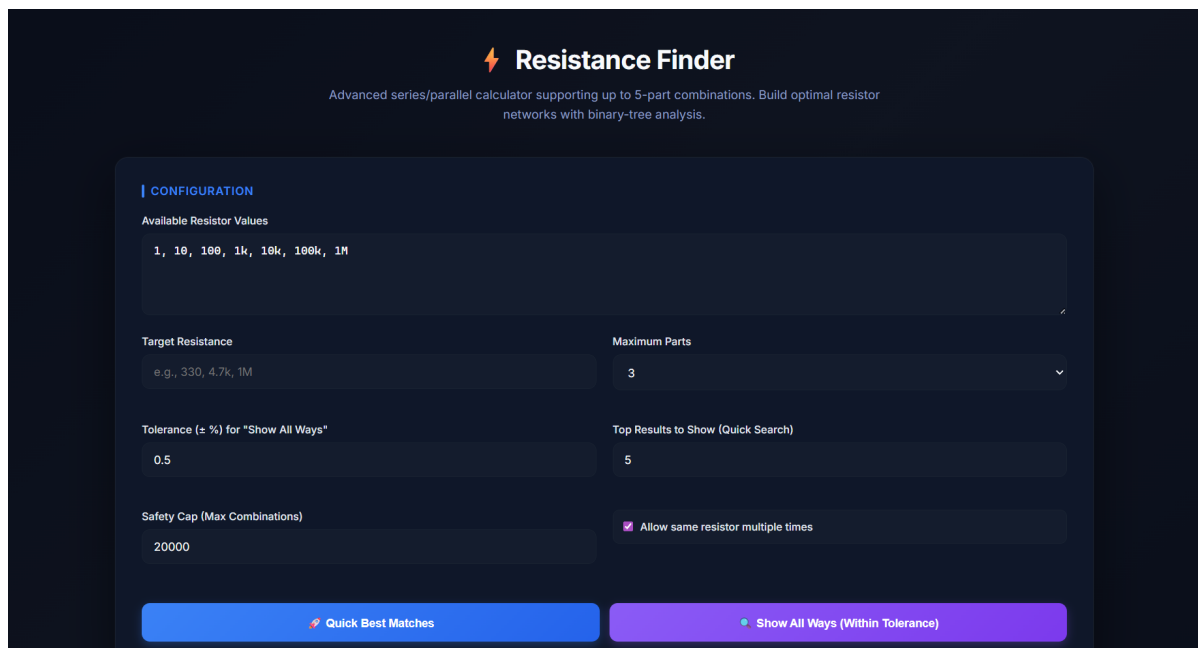
to get the values of the resistors we will use for these weights and biases we need to divide the value of $R_{ref} = 10k\Omega$ over each one, and the value will be the resistor desired.

6

Resistance Finder

6.1. Resistance Finder website

all the values of each resistor is not available in the market, but we can connect the available resistors in a way that gives us the value desired. So I asked my friend (Abdallah Qusit) to make me a program that automate this process. and he made me the website shown in Figure[6.1]



The screenshot shows the 'Resistance Finder' website interface. At the top, there is a lightning bolt icon and the title 'Resistance Finder'. Below the title, a subtitle reads: 'Advanced series/parallel calculator supporting up to 5-part combinations. Build optimal resistor networks with binary-tree analysis.' The main configuration area is titled 'CONFIGURATION' and contains several input fields and buttons. The 'Available Resistor Values' field is a text input containing '1, 10, 100, 1k, 10k, 100k, 1M'. The 'Target Resistance' field is a text input containing 'e.g., 330, 4.7k, 1M'. The 'Maximum Parts' field is a dropdown menu set to '3'. The 'Tolerance (± %) for "Show All Ways"' field is a text input containing '0.5'. The 'Top Results to Show (Quick Search)' field is a text input containing '5'. The 'Safety Cap (Max Combinations)' field is a text input containing '20000'. There is a checkbox labeled 'Allow same resistor multiple times' which is checked. At the bottom, there are two buttons: 'Quick Best Matches' (blue) and 'Show All Ways (Within Tolerance)' (purple).

Figure 6.1: <https://resistance-finder.netlify.app>

6.2. equivalent resistors connections

after calculating the values of the resistors that represents the weights and biases, I used the website above to create each equivalent circuit.

Resistor	value	equivalent circuit	error $\pm$ %
Rref1-1	10k	10k	0
R 1-1-1	5471.9	((1.0000 k Ω + 27.0000 k Ω) 6.8000 k Ω)	735.632 m Ω (0.013444%)
R 1-1-2	5461.6	(560 Ω + (1.0000 k Ω + 3.9000 k Ω))	1.6 Ω (0.029295%)
R 1-1-3	5460.2	(560 Ω + (1.0000 k Ω + 3.9000 k Ω))	200.000 m Ω (0.003663%)
R 1-1-4	52205	(470 Ω + (4.7000 k Ω + 47.0000 k Ω))	35 Ω (0.067043%)
R 1-1-5	5469.1	(470 Ω + (10.0000 k Ω 10.0000 k Ω))	900.000 m Ω (0.016456%)
Rof 1-1	6284.9	(3.3000 k Ω + (4.7000 k Ω 8.2000 k Ω))	2.6969 Ω (0.042911%)
Rref1-2	10k	10k	0
R 1-2-1	4437.5	(390 Ω + (6.8000 k Ω 10.0000 k Ω))	119.048 m Ω (0.002683%)
R 1-2-2	4437.5	(390 Ω + (6.8000 k Ω 10.0000 k Ω))	119.048 m Ω (0.002683%)
R 1-2-3	4437.5	(390 Ω + (6.8000 k Ω 10.0000 k Ω))	119.048 m Ω (0.002683%)
R 1-2-4	4437.5	(390 Ω + (6.8000 k Ω 10.0000 k Ω))	119.048 m Ω (0.002683%)
R 1-2-5	4437.5	(390 Ω + (6.8000 k Ω 10.0000 k Ω))	119.048 m Ω (0.002683%)
Rof 1-2	2531.2	((3.3000 k Ω + 3.9000 k Ω) 3.9000 k Ω)	1.4703 Ω (0.058086%)
Rref2-1	10k	10k	0
R 2-1-1	2985.4	((2.7000 k Ω + 10.0000 k Ω) 3.9000 k Ω)	1.6651 Ω (0.055773%)
R 2-1-2	1231.3	((470 Ω 3.3000 k Ω) + 820 Ω)	105.836 m Ω (0.008595%)
Rof 2-1	5885.6	((4.7000 k Ω + 39.0000 k Ω) 6.8000 k Ω)	1.2436 Ω (0.021129%)
Rref2-2	10k	10k	0
R 2-2-1	2028.4	((270 Ω + 3.3000 k Ω) 4.7000 k Ω)	499.637 m Ω (0.024632%)
R 2-2-2	7653.9	((680 Ω + 10.0000 k Ω) 27.0000 k Ω)	1.0338 Ω (0.013506%)
Rof 2-2	8583	((390 Ω 27.0000 k Ω) + 8.2000 k Ω)	1.4469 Ω (0.016857%)
Rref2-3	10k	10k	0
R 2-3-1	2856.6	((680 Ω + 10.0000 k Ω) 3.9000 k Ω)	190.123 m Ω (0.006656%)
R 2-3-2	40771	((3.3000 k Ω 3.9000 k Ω) + 39.0000 k Ω)	16.5 Ω (0.040470%)
Rof 2-3	7694	((1.0000 k Ω + 8.2000 k Ω) 47.0000 k Ω)	49.822 m Ω (0.000648%)
Rref2-4	10k	10k	0
R 2-4-1	3910.7	(390 Ω + (820 Ω + 2.7000 k Ω))	700.000 m Ω (0.017900%)
R 2-4-2	6964.6	((330 Ω 330 Ω) + 6.8000 k Ω)	400.000 m Ω (0.005743%)
Rof 2-4	1738.6	((820 Ω + 1.0000 k Ω) 39.0000 k Ω)	253.503 m Ω (0.014581%)

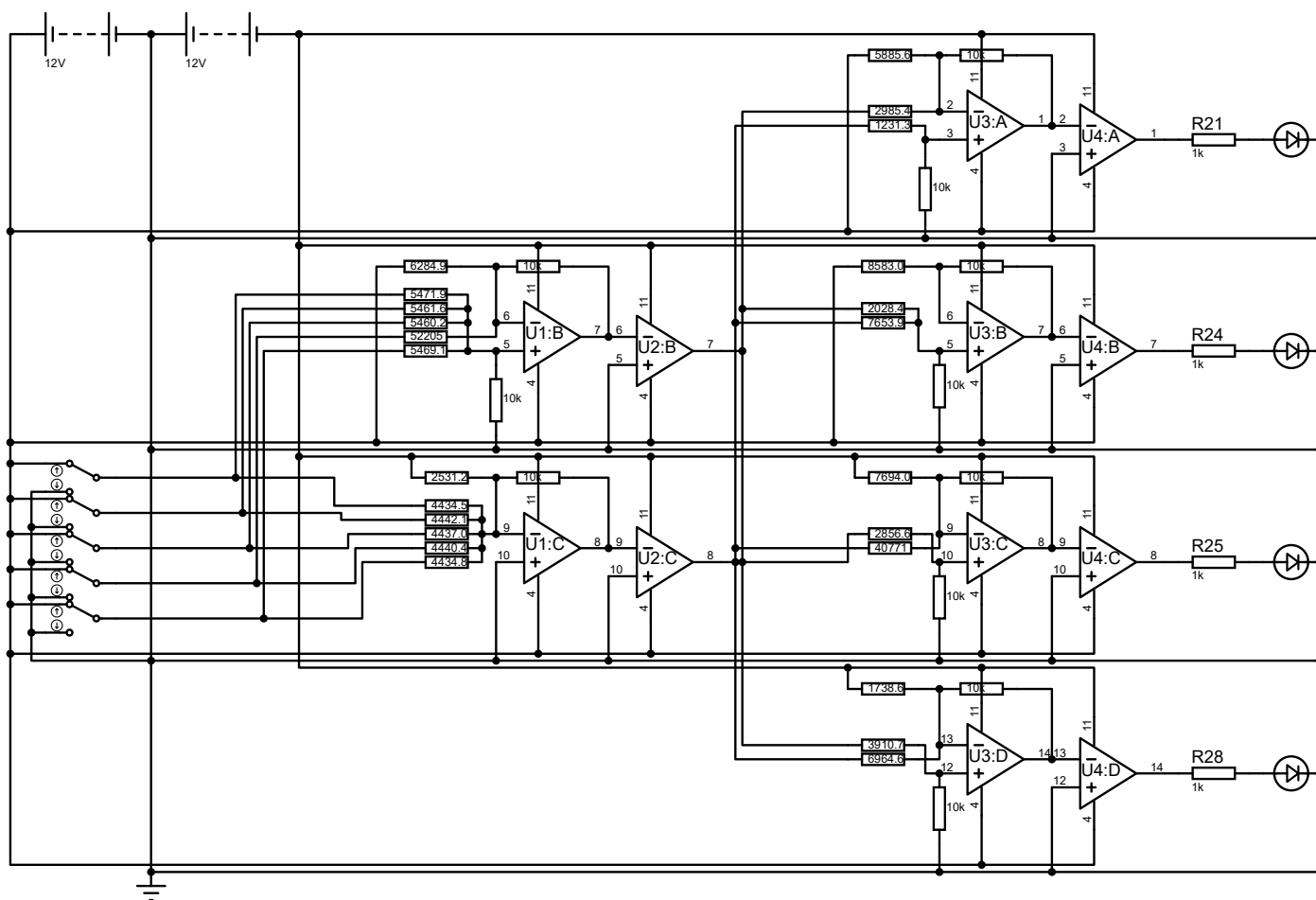
Figure 6.2

The first column is the resistors name, which in the way (R-layer-neuron-resistor) for the regular weights, and (Rof-layer-neuron) for bias resistor, and (Rref-layer-neuron) for the reference resistor. The second column is the value of the resistor and it's equal $\frac{R_{ref}}{w_{l-n-r}}$. The third column is the equivalent connection of the resistors available in the market to produce the desired resistance. The last row is the error caused by the connection and it's very low so we can ignore it.

7

Hardware Re-implementation

7.1. schematic



7.2. layout

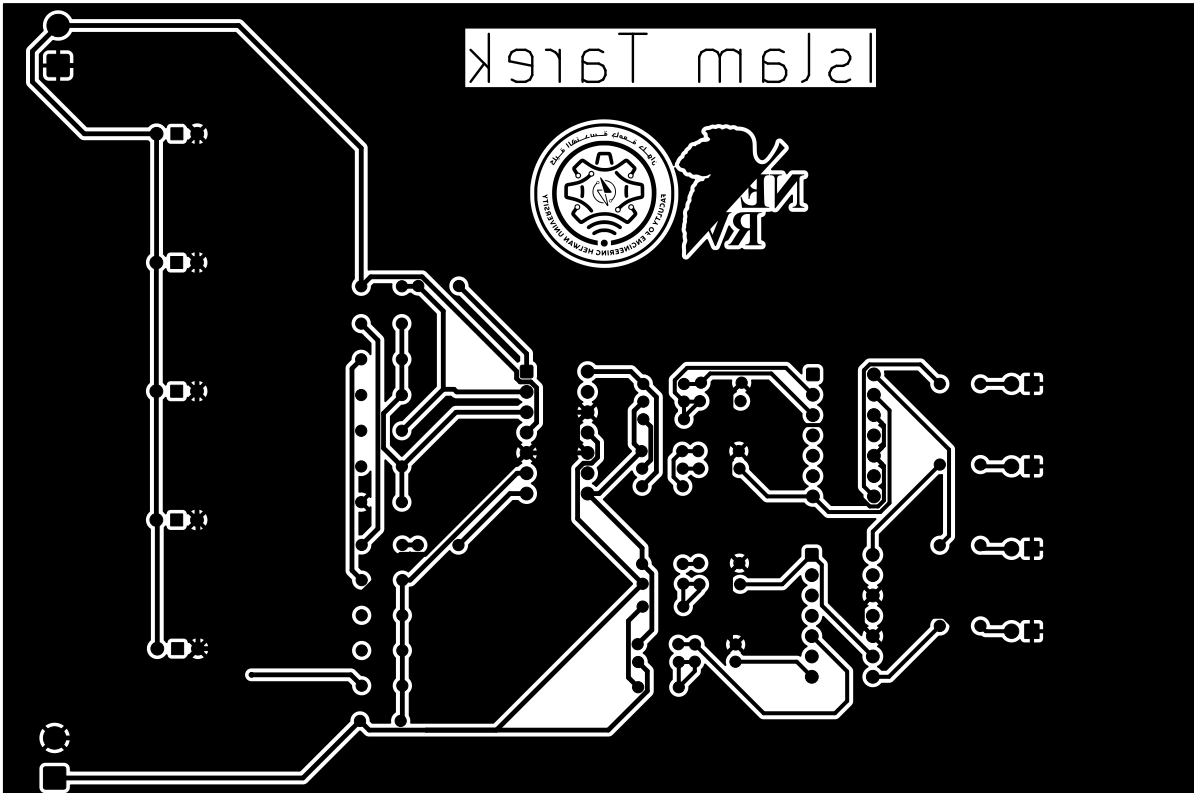


Figure 7.1

7.3. KiCAD interface

I will connect the red junctions with a regular wires.

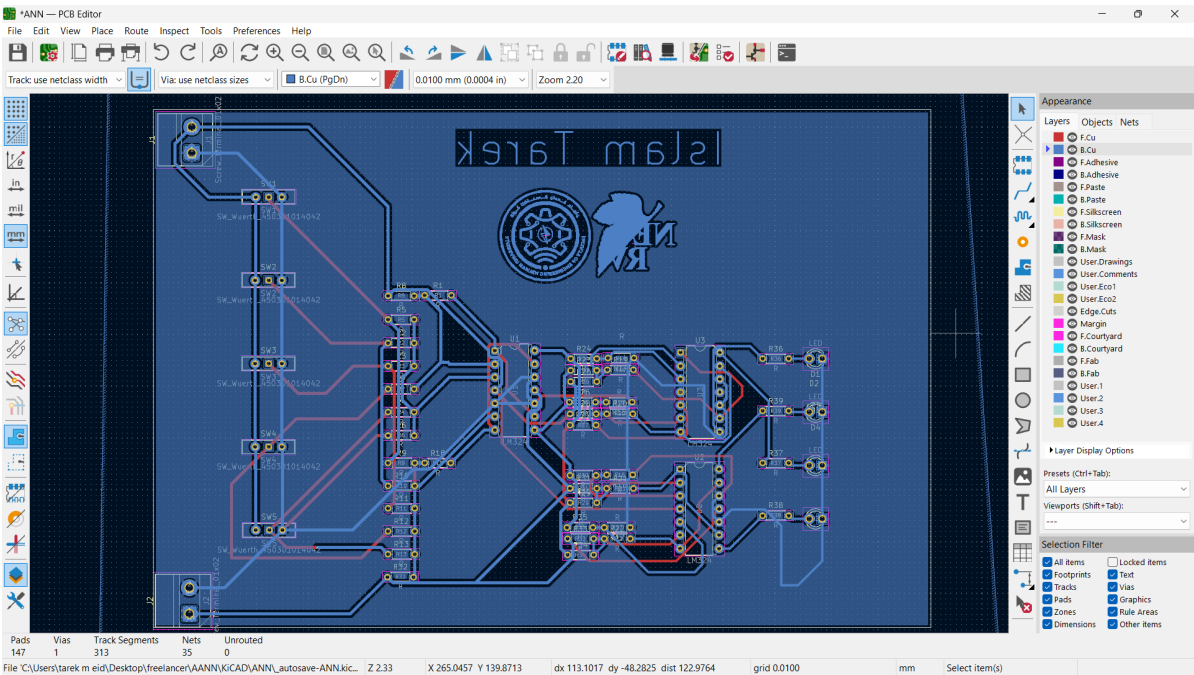


Figure 7.2

7.4. bottom layer

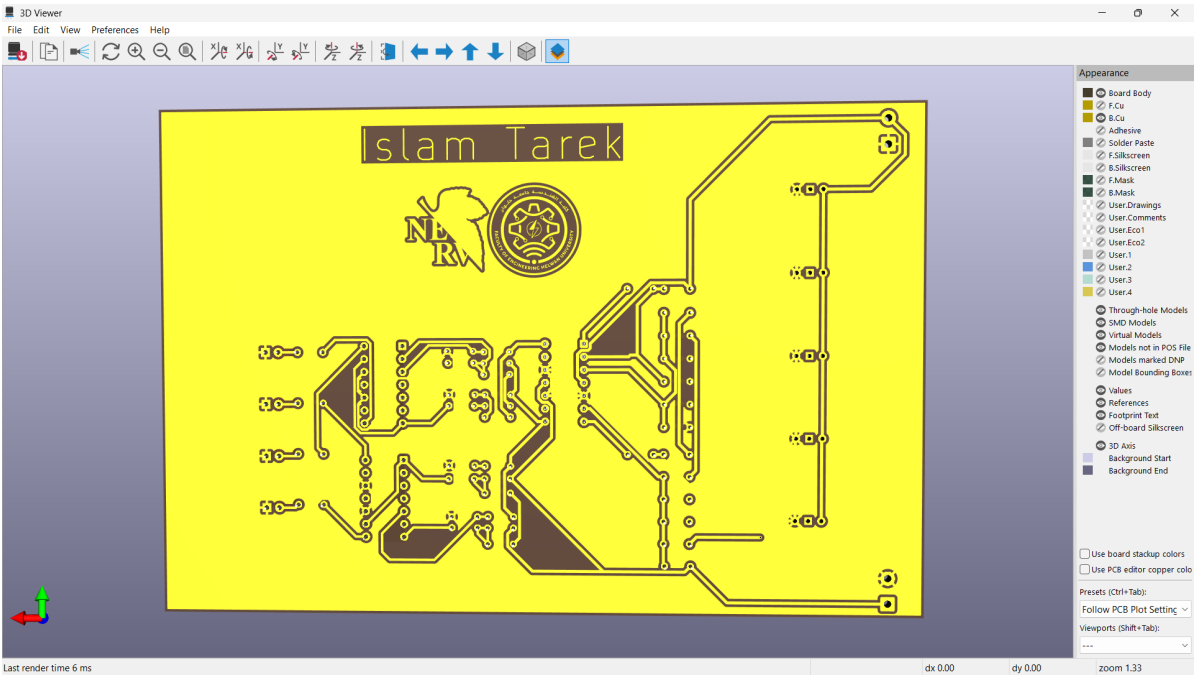


Figure 7.3

7.5. Isometric View

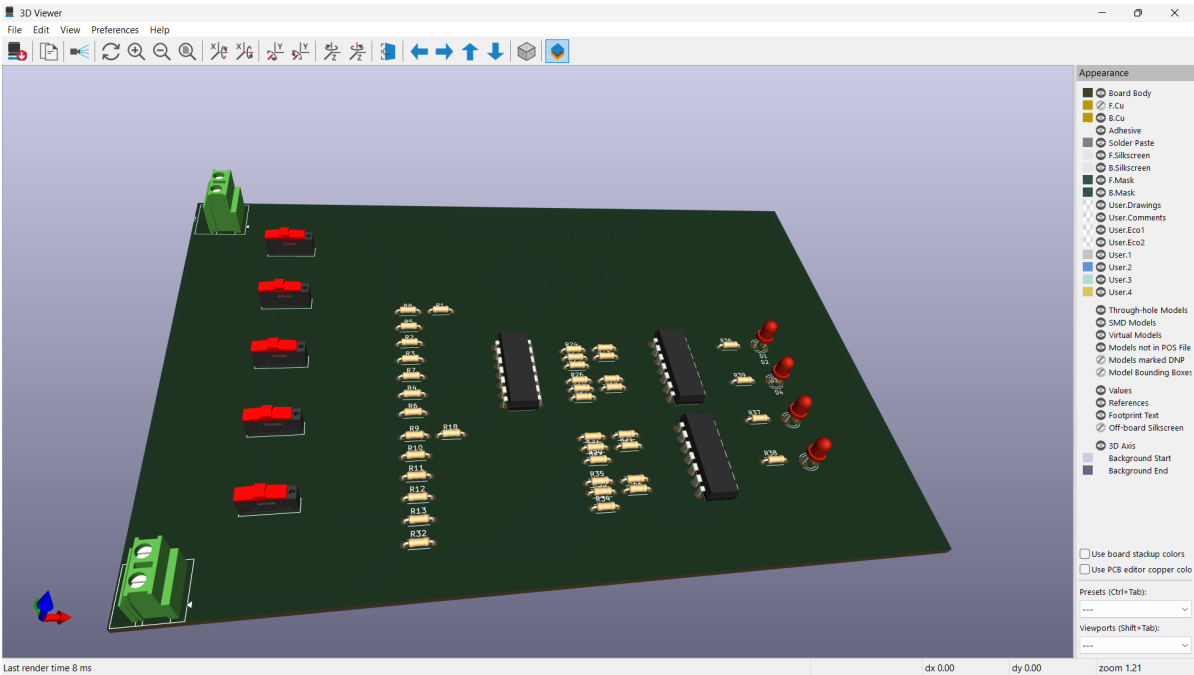
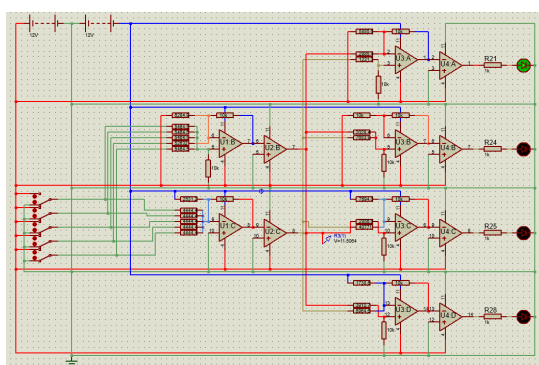


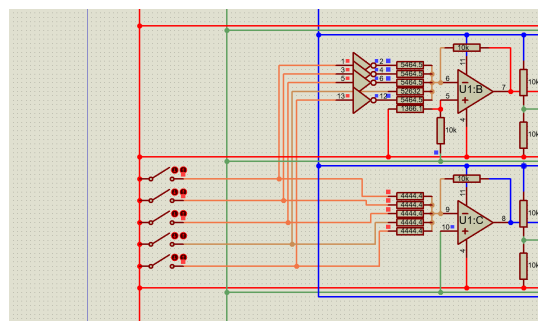
Figure 7.4

8

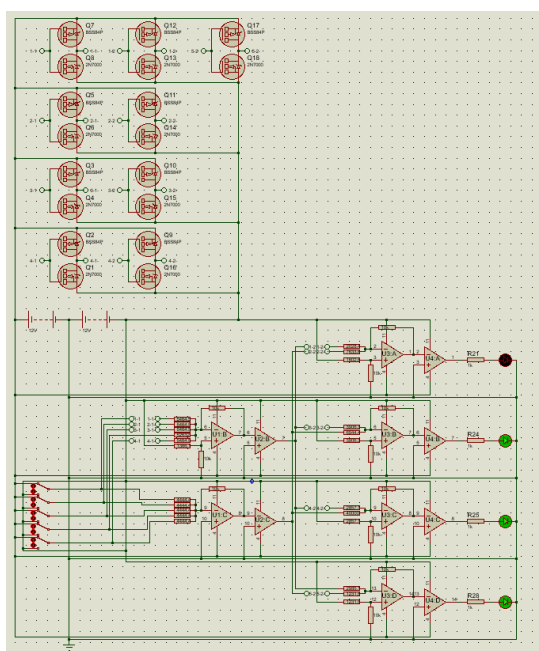
Gallery



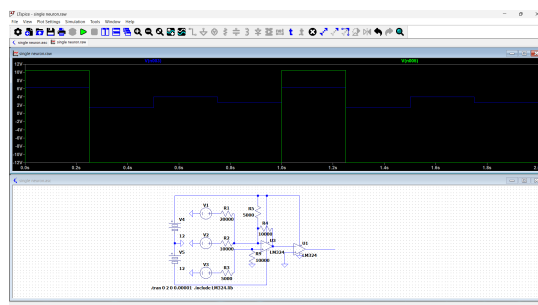
(a) Image 1



(b) Image 2



(c) Image 3



(d) Image 4

9

Conclusion

Finally, I would like to thank my friends from the Computer and Systems Engineering Department, Abdullah Qasit and Muhammad Imad. The project would not have been possible without Muhammad

Imad's explanation of neural networks and Abdullah Qasit's program.

This project was intended to train participants in digital medical vital signs readings, but the project's timeframe wasn't sufficient for all the work.

I intend to complete the project on my own, but that's unlikely to happen.

I hope you appreciate the effort put into this project.